

Sit!!!



The Command Pattern

Human Computer Interaction Research
University of Nevada, Reno



Command

- Behavioral Patterns
 - » strategy
 - » observer
 - » **command**
- Structural Patterns
 - » decorator
- Creational Patterns
 - » factory method
 - » abstract factory
 - » singleton

Problem

“your program has many different actions it can perform, implementing these would lead to huge if-elseif or switch blocks”.

Dog Training



Roll



Bark



Sit

Naive solution

```
public static void main(String[] args) {  
    Dog calvin = new Dog("Snoop");  
    if (args[0].equals("bark")) {  
        calvin.bark(); }  
    else if (args[0].equals("bite")) {  
        calvin.bite(); }  
    else if (args[0].equals("roll")) {  
        calvin.rollover();  
    }  
}
```

should this even
be in main?

Works but gets
messy when you
add lots of
commands
(bite, attack,
eat homework,
barf...)

Command Pattern

The Command Pattern:
encapsulates a request as an
object, thereby letting you
parameterize other objects with
different requests, queue or log
requests, and support undoable
operations

Command Interface

```
public interface DogCommand {  
    public void execute();  
}
```

- ☐ You can put any “generic” method in here:
 - ☐ log()
 - ☐ undo()
 - ☐ delete()
 - ☐ load()

Specific Commands

```
public class BarkCommand implements DogCommand {  
    public void execute() {  
        System.out.println("Bow wow wow!!");  
    }  
}
```

```
public class BiteCommand implements DogCommand {  
    public void execute() {  
        System.out.println("Yippee yo yippee yay");  
    }  
}
```


Dog

```
public class Dog {  
    String name;  
    DogCommand commands[] = new DogCommand[2];  
  
    public Dog(String name) {  
        this.name=name;  
        commands[0] = new SitCommand();  
        commands[1] = new BarkCommand();  
    }  
}
```

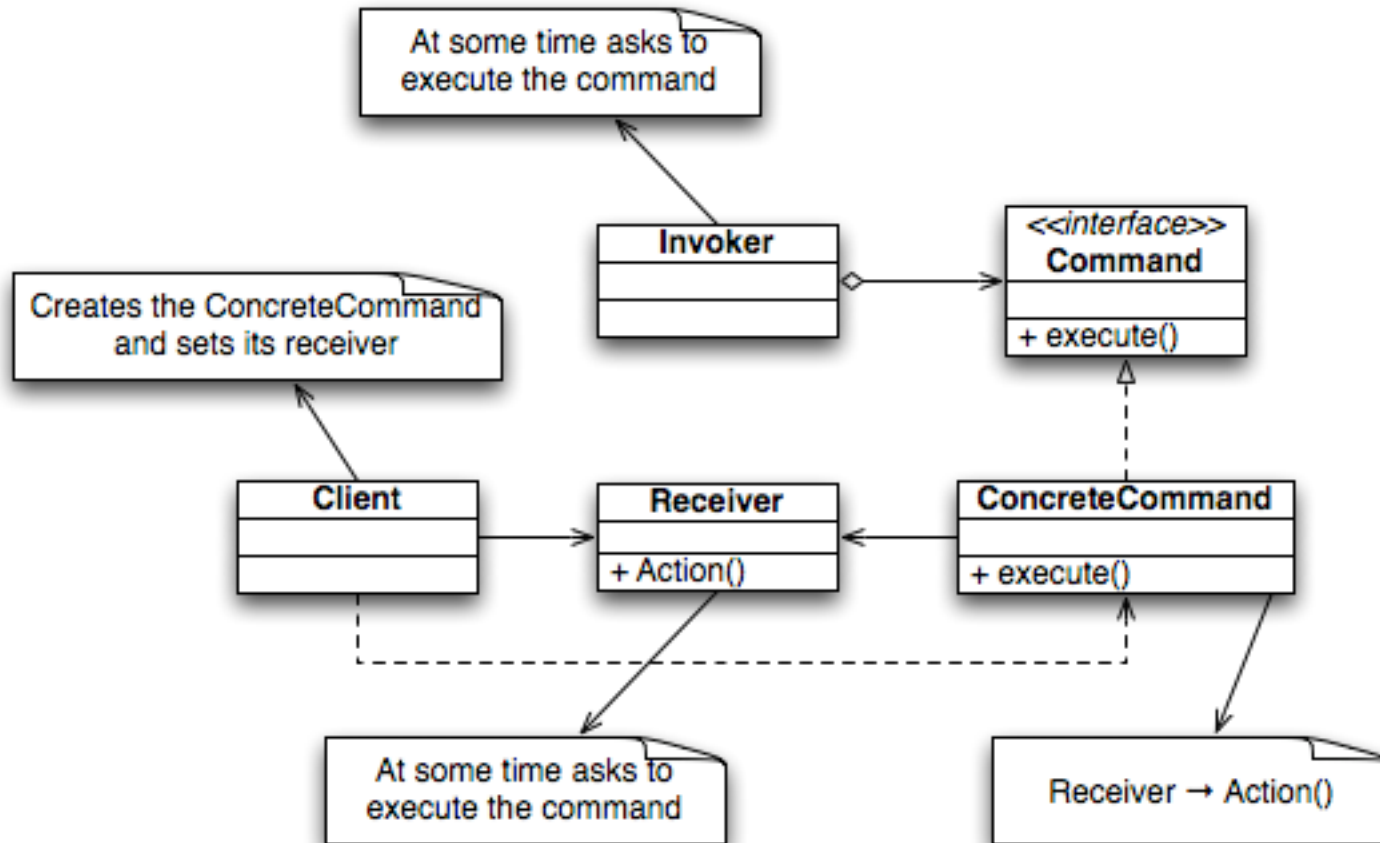
Main class

```
public class TestDogCommand {  
    public static void main(String[] args) {  
        DogCommand cmd;  
        try {  
            cmd = (DogCommand) Class.forName(args[0]).newInstance();  
            cmd.execute();  
        } catch (InstantiationException e) {  
            e.printStackTrace();  
        } catch (IllegalAccessException e) {  
            e.printStackTrace();  
        } catch (ClassNotFoundException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

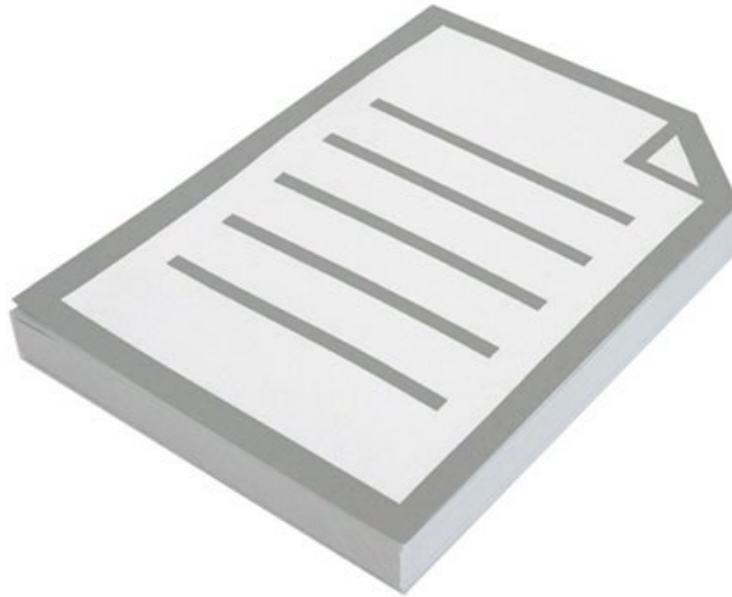
OK this is ugly,
but it scales up a
lot better

```
Eelke-Folmers-MacBook-Pro:bin eelke$ java TestDogCommand BarkCommand  
Bow wow wow!!
```

Class Diagram

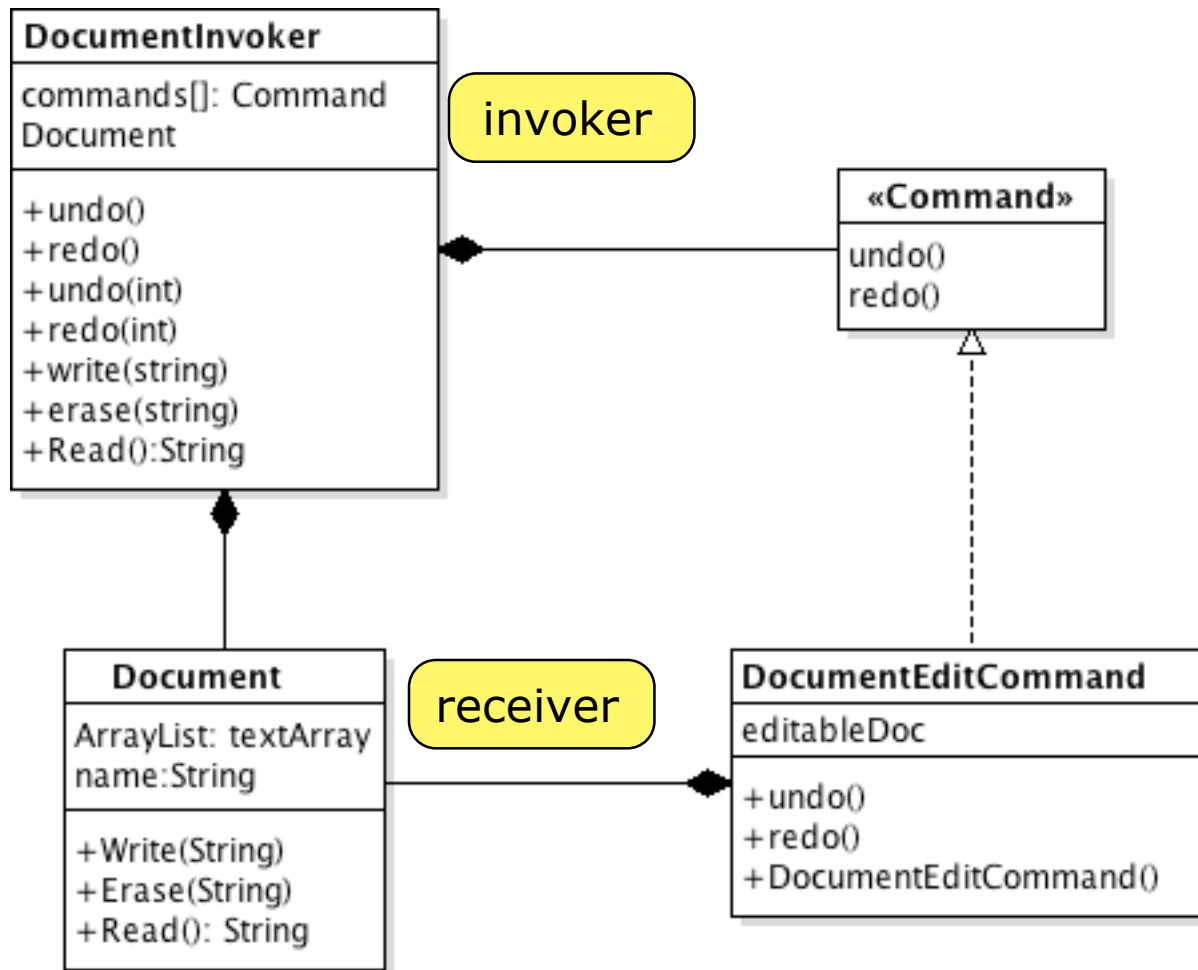


Example



Document Editor

Class diagram



Command Interface

```
public interface Command {  
    public void undo();  
    public void redo();  
}
```

Document write Command

```
public class DocumentWriteCommand implements Command {  
    private Document editableDoc;  
    private String text;  
  
    public DocumentWriteCommand(Document doc, String text) {  
        this.editableDoc = doc;  
        this.text = text;  
        editableDoc.Write(text);  
    }  
  
    public void undo() {  
        editableDoc.Erase(text);  
    }  
  
    public void redo() {  
        editableDoc.Write(text);  
    }  
    public String toString(){  
        return "Write";  
    }  
}
```

Document Invoker

```
import java.util.ArrayList;

public class DocumentInvoker {
    private ArrayList commands = new ArrayList();
    private Document currentdoc;

    public DocumentInvoker(String name) {
        currentdoc = new Document(name);
    }

    public void Redo(int level) {
        Command tmp;
        System.out.println("---- Redo level " + level);
        tmp = (Command) commands.get(level);
        tmp.redo();
    }

    public void Undo(int level){
        Command tmp;
        System.out.println("---- Undo level " + level);
        tmp = (Command) commands.get(level);
        tmp.undo();
    }
}
```


Document Invoker

```
public void Write(String text) {
    DocumentWriteCommand cmd = new DocumentWriteCommand(currentdoc, text);
    commands.add(cmd);
}

public void Bold(int line) {
    DocumentBoldCommand cmd = new DocumentBoldCommand(currentdoc, line);
    commands.add(cmd);
}

public void RemoveBold(int line) {
    DocumentRemoveBoldCommand cmd = new DocumentRemoveBoldCommand(currentdoc, line);
    commands.add(cmd);
}

public String Read() {
    return currentdoc.Read();
}

public void printCommandBuffer() {
    StringBuffer txt = new StringBuffer();
    txt.append("---- Commands issued----\n");
    for (int i = 0; i < commands.size(); i++) {
        txt.append(i + ":" + (Command) commands.get(i) + "\n");
    }
    System.out.println(txt.toString());
}
}
```



```
public class TestEditor {  
    public static void main(String[] args) {  
        DocumentInvoker myDocument = new DocumentInvoker ("message_to_rudi");  
  
        myDocument.Write("Hey! I just met you...");  
        myDocument.Write("and this is crazy, ");  
        myDocument.Write("but here's my number");  
        myDocument.Write("so call me maybe?");  
  
        myDocument.Bold(0);  
        myDocument.Undo(1);  
  
        myDocument.printCommandBuffer();  
  
        System.out.println(myDocument.Read());  
    }  
}
```